Université de Caen Basse-Normandie

Projet annuel de Master 1

Maintien de la cohésion dans les algorithmes de nuées

Auteur :
Guillaume Desquesnes

Encadrant :
Grégory Bonnet

2013-2014

Table des matières

1 Introduction **1**

2 État de l'art **3**
2.1 Algorithmes de nuées 3
2.2 Vivaldi : un algorithme de cartographie distribuée 5
2.3 MaDKit 6

3 Réalisation **8**
3.1 Application 8
3.2 Conception 9
3.3 Point de vue agent 10

4 Tests et résultats **11**
4.1 Cadre expérimental 11
4.2 Métriques 11
4.3 GPS non bruité 12
4.4 GPS bruité 13
4.5 Algorithme de Vivaldi 14
4.6 Influence de la fréquence de communication 15
4.7 Vivaldi précédé par un GPS 16

5 Conclusion **18**
5.1 Problèmes 18
5.2 Résultats 18
5.3 Limites du projet 18
5.4 Perspectives 18
5.5 Bilan 19

Annexes

A Mode d'emploi **20**
A.1 Lancement 20
A.2 Options 20
A.3 Interface graphique 21

B Diagramme UML **23**

Bibliographie **24**

Table des figures

1.1 Drones se déplacant en nuée 1

2.1 Cohésion, séparation et alignement 3
2.2 Modèle métrique et topologique 4
2.3 Pseudo code de l'algorithme de Vivaldi 5
2.4 Exemple d'execution de l'algorithme de vivaldi 6
2.5 Modèle AGR 7

3.1 Fonctionnement d'un agent 10

4.1 GPS non bruité : cohésion et collisions 12
4.2 GPS bruité : cohésion et collision 13
4.3 Vivaldi : cohésion et collisions 14
4.4 Comparaison des collisions et cohésions entre GPS et Vivaldi 15
4.5 Influence de la fréquence de communication sur le GPS et Vivaldi 16
4.6 Algorithme de Vivaldi précédé par un GPS. 16

A.1 Fenêtre principale de l'application 21
A.2 Exemple d'execution de l'algorithme de vivaldi 22

B.1 Diagramme UML de l'application 23

1 Introduction

De nos jours, il existe de plus en plus de robots devant agir en groupe, coopérer pour atteindre un objectif, ou plus simplement se répartir équitablement des tâches à accomplir.

On peut citer par exemple, les robots aspirateur ou robots tondeuse qui ont pour objectif de nettoyer, ou tondre une zone prédéfinie de la manière la plus efficace possible, en évitant donc de passer plusieurs fois sur une même zone, tout en évitant les collisions avec leur environnement ou avec d'autres robots.

De même pour les *UAV* (Unmanned aerial vehicle) ou drones, dont certaines missions consistent à surveiller des régions particulières. Ils peuvent être contrôlés par un être humain, ou être autonomes. Une des fonctions des UAV autonomes est de se déplacer en nuée [4] de manière similaire à un vol d'oiseau.

Figure 1.1 – Drones se déplacant en nuée
`http://gracegao.ae.illinois.edu/`

Pour se déplacer de cette manière, ces drones utilisent un GPS afin de se repérer dans l'espace, ainsi que divers capteurs permettant de repérer, détecter et positionner dans l'espace les autres drônes proches pour se déplacer ensembles de manière ordonnée, à la façon d'une nuée. On peut se demander ce qui se passe lorsque les GPS ne fonctionnent plus ou ne sont pas utilisables. Ou encore comment chercher à diminuer l'utilisation du GPS afin de réduire la consommation d'énergie du drône.

Pour cela il existe des moyens, tel que l'algorithme Vivaldi [1], permettant de générer une carte artificielle à partir d'entités statiques, immobiles ne possèdant pas d'informations pertinantes sur la localisation des drones, mais pouvant communiquer les unes avec les autres.

Nous allons donc observer l'entremêlement de ces algorithmes, de déplacement et de positionnement afin de savoir si cet entremêlement permet à ces drônes de continuer à se déplacer en groupe et si oui, dans quelles conditions.

Dans la première partie, nous présenterons plus en détails les algorithmes utilisés ainsi que la plate-forme de développement utilisée. Puis en deuxième partie sera présentée l'application qui a été développée lors de ce projet. La troisième et dernière partie présentera les tests effectués ainsi que leurs analyses.

2 | État de l'art

2.1 Algorithmes de nuées

Les algorithmes de nuéees sont des algorithmes décentralisés, de déplacement d'agents, des entités autonomes, visant à reproduire les comportements de nuée (ou essaim voire banc), présents dans le règne animal, aussi bien chez les oiseaux que les poissons, les insectes ou encore chez les mammifères vivant en meute voire même chez certaines plantes. Le comportement en nuée est un comportement collectif d'animaux de taille similaire qui se regroupent afin de se déplacer, ou de migrer dans une direction commune.

Ce type de comportement a été simulé [3] pour la première fois par Craig W. Reynolds, qui a reproduit artificiellement une nuée d'oiseau avec son programme `Boids`. Un algorithme de nuée est composé de trois étapes, que chaque entité effectuera à chaque tic d'horloge, chaque pas de temps de l'entité.

Elles calculeront trois vecteurs de manière à :
- se rapprocher de leurs voisins (cohésion) ;
- s'éloigner des voisins trop proche (séparation) ;
- s'orienter dans la même direction que leurs voisins (alignement).

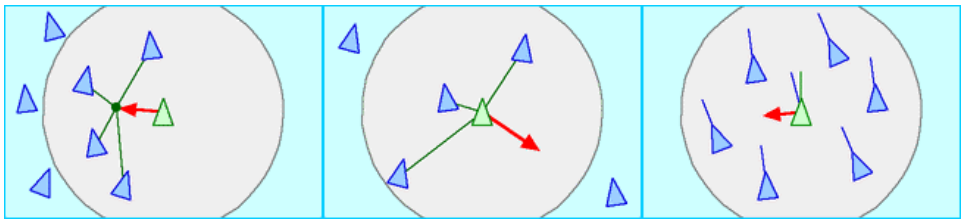

Figure 2.1 – Cohésion, séparation et alignement

Il existe plusieurs façons de déterminer les voisins de chaque entité. Elle peut soit choisir :

– l'ensemble des voisins situés dans une zone autour de l'entité (distance métrique).
– les n plus proches voisins (distance topologique) ;

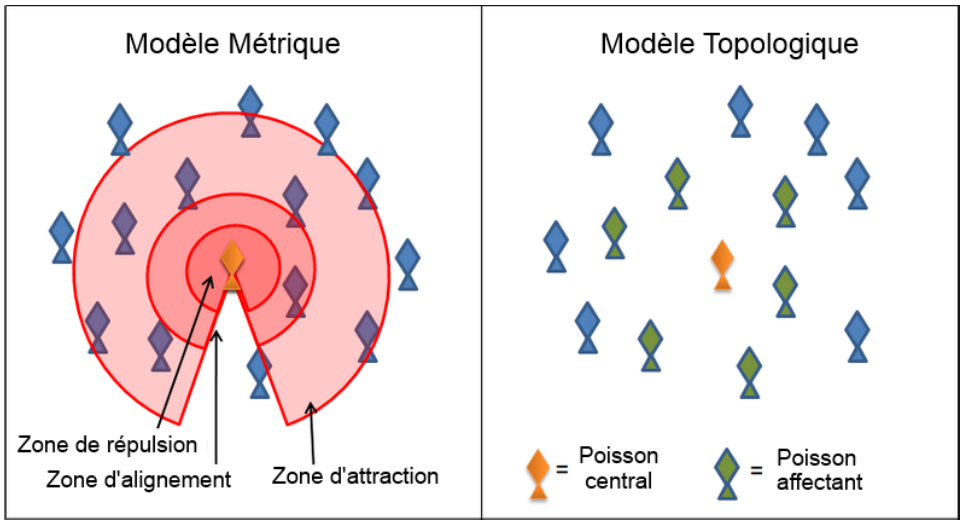

FIGURE 2.2 – Modèle métrique et topologique

L'algorithme de nuée calculera un vecteur $\vec{v}$ de déplacement pour l'entité grâce à ces informations.

Soit e la position de l'entité, N l'ensemble des positions des voisins de cette entité, $S \subseteq N$ l'ensemble des voisins trop proches et O l'ensemble des vecteurs de déplacement ou d'orientation des voisins.

– le vecteur de cohésion $\vec{c} = \sum_{p \in N} \vec{ep}$;
– le vecteur de séparation $\vec{s} = \sum_{p \in S} \vec{ep}$;
– le vecteur d'alignement $\vec{a} = \sum_{\vec{u} \in O} \frac{\vec{u}}{\|\vec{u}\|}$.

Le vecteur obtenu pour le déplacement de l'entité sera une somme pondérée des trois vecteurs ci-dessus.

$$\vec{v} = \alpha\vec{c} + \beta\vec{s} + \gamma\vec{a} \tag{2.1}$$

Où les poids (α, β, γ) seront à choisir selon les cas. Dans le cadre de ce projet les valeurs sont $(1.5, 2, 1)$, ce qui donne plus d'importance au vecteur de séparation, car il est préférable d'éviter au maximum les collisions.

2.2 Vivaldi : un algorithme de cartographie distribuée

L'algorithme Vivaldi [1] a pour but de mettre en place, de manière automatique et décentralisée, un système de coordonées artificielles pour un ensemble d'entité fixe, comme par exemple les nœuds d'un réseau informatique.

La cartographie d'un réseau permet à chaque noeud d'estimer le temps de latence dans les communications du réseau et de faciliter la recherche d'un plus court chemin de communication entre deux nœuds n'étant pas reliés directement.

La construction d'un système de coordonées artificielles se déroule de manière itérative. Au départ, chaque entité possède une position supposée dans l'environnement.

À chaque tic d'horloge (locale), chaque agent récupère les positions des autres agents à portée de communication. Pour chaque position reçue, l'agent calcule un vecteur de correction en fonction de celle ci, là où l'autre agent pense se situer, et le temps de latence dans la communication, la moitié du temps entre la demande et la réponse. La somme de tous ces vecteurs déterminera la nouvelle position supposée de l'agent.

Données : x les coordonnées supposées de l'agent et δ un poids de l'algorithme.
Début :
  broadcast(demande de position)
  $\vec{f} = (0, 0)$
  pour toute réponse r **faire**
    //Calcul du vecteur entre les agents
    $\vec{v} = (x - r.x)$
    //Calcul de l'erreur
    $e = r.rtt - \|\vec{v}\|$
    //Calcul de la direction causée par l'erreur
    $\vec{f} = \vec{f} + \hat{\vec{v}} \times e$
  fin pour
  $x = \delta \times \vec{f}$
Fin

FIGURE 2.3 – Pseudo code de l'algorithme de Vivaldi

Lorsque les entités sont fixes, l'algorithme convergera vers une cartographie correcte, qui sera une rotation et ou une translation de la cartographie réelle.

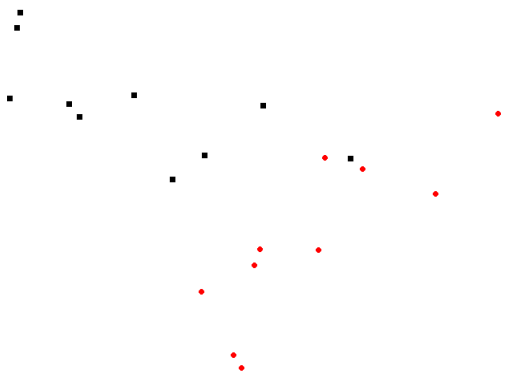

FIGURE 2.4 – Exemple d'execution de l'algorithme de vivaldi
Les carrés représentent les positions réelles et les ronds celles générées par l'algorithme.

Nous souhaitons savoir si cette algorithme peut être adapté à des entités mobiles et si oui sous quelles conditions.

2.3 MaDKit

MaDKit [2] (Multiagent Development Kit) est une plate-forme multi-agents, libre (GNU GPL) et écrite en java, construite en respectant le modèle Agent/Groupe/Rôle [5] (AGR), voir figure 2.5.

Un **agent** est une entité active jouant un rôle à l'intérieur d'un groupe. Chaque agent peut avoir plusieurs rôles et appartenir à un ou plusieurs groupes.

Un **groupe** est un ensemble d'agents possèdant des caractèristiques communes et un même contexte pour leurs activités. Les agents ne peuvent communiquer entre eux que s'ils font parti d'un même groupe.

Un **rôle** est la représentation d'un attribut (status, fonction, position) d'un agent dans un groupe. Les rôles sont locaux à chaque groupe et un même rôle peut être affecté à plusieurs agents.

Cette plate-forme permet de développer rapidement une application comportant plusieurs agents pouvant communiquer simplement entre eux, n'uti-

lisant pas de langage de communication particulier. Les agents communiquent entre eux en suivant le modèle AGR en s'échangeant des instances d'objet. De plus MaDKit fournit des outils permettant facilement le calcul de métriques sur les agents, ainsi que la visualisation des agents et de leurs métriques.

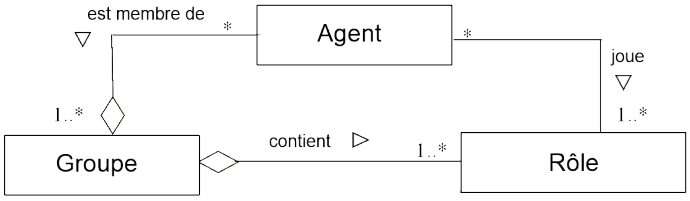

FIGURE 2.5 – Modèle AGR

3 | Réalisation

3.1 Application

Pour ce projet, nous avons réalisé une application, développée en Java permettant de visualiser l'évolution d'un certain nombre d'agents, dans un environnement clos, essayant de se déplacer en nuée.

Les agents de cette application se déplacent dans un environnement fermé et évitent les murs en changeant de direction lorsqu'ils en sont trop près. Si l'environnement est fermé, c'est afin de faciliter la création des premières nuées, d'éviter que des groupes d'agents se repoussant finissent par se rejoindre si l'environnement était torique, mais aussi pour des facilités de conceptions.

Lors du fonctionnement de l'application, chaque agent se déplacera grâce à l'algorithme de nuée, en utilisant pour se positionner un GPS plus ou moins précis ou l'algorithme de Vivaldi. Les agents partagent tous le même algorithme de positionnement, celui-ci pouvant changer lors de l'éxecution. À chaque tic d'horloge globale, des métriques sont calculées et sauvegardées dans un fichier avec les informations sur les différents paramètres utilisés à cet instant. Plus d'informations sur ces métriques seront apportées dans la partie suivante.

L'application possède une grande quantité de paramètres à passer sur la ligne de commande, afin de spécifier le déroulement de l'application. À ce moment, la ligne de commande est la seule façon d'indiquer la majorité des paramètres

Les principales options au lancement sont :

- la présence ou non d'une interface graphique
- le nombre d'agents initiaux ;
- la durée d'exécution ;
- l'algorithme de positionnement utilisé.

les autres options sont disponibles en annexe dans le mode d'emploi.

L'utilisation d'une interface graphique permet de visualiser en temps réel

la position des agents dans l'environnement, ainsi que de pouvoir modifier certains paramètres en cours d'exécution, comme le nombre d'agent, le système de positionnement ou encore la précision du GPS. De plus, l'évolution des métriques au fur et à mesure du temps sera affichée dans une fenêtre.

Son absence permet de rendre plus rapide l'exécution de l'application, en diminuant l'écart de temps entre deux tics d'horloge, cela est favorable pour effectuer des séries de test.

3.2 Conception

Le peu de documentation, sur l'utilisation de la plate-forme MaDKit, a rendu la conception de ce projet plus complexe. La prise en main de MaDKit s'est donc fait par la compréhension et l'analyse des exemples fournis par le site officiel [2]. Quelques problèmes liés au fonctionnement de la plate-forme ont pu être contournés grâce à la lecture du code source de la plate-forme.

Notre application est séparée en plusieurs parties, dont chacune de leurs sous-parties sont des agents MaDKit, tous membres d'un même groupe leur permettant de communiquer ensemble :

- les agents mouvant ;
- les utilitaires (métriques, scheduler, lanceur) ;
- l'interface graphique (agents mouvant, métriques).

Les interfaces graphique, les utilitaires et les agents mouvant sont tous des agents selon MaDKit, appartenant à des rôles bien précis afin de faciliter les communications. Les relations entre ces différents agents est visible sur le diagramme UML de l'application disponible en annexe (voir figure B.1).

La plate-forme MaDKit requiert, au lancement, un agent principal, un `lanceur` (ou `Launcher`), qui devra initialiser les paramètres de fonctionnement de l'application. De plus, il devra créer (ou lancer), tout autre agent devant être instancié au commencement de l'application. Cet agent a donc pour unique rôle de traîter la ligne de commande, l'ajout et la suppression d'agents et la fermeture de l'application.

Afin de simuler une horloge locale, les agents ont besoin d'un `scheduler`, afin de déclencher, de manière régulière, une méthode ciblée d'un agent en fonction de son rôle. Tous les agents, autre que le lanceur, sont affectés par les scheduler afin de se déplacer, de mesurer (pour les métriques), ou encore d'actualiser l'affichage (pour l'interface graphique).

De manière similaire, les agents graphiques utilisent le rôle des agents qu'ils doivent représenter, afin de récupérer l'information à visualiser.

3.3 Point de vue agent

Chaque agent effectue les mêmes taches en suivant un ordre bien précis.

En premier lieu, il envoie via broadcast une message demandant à tout agent le reçevant de renvoyer sa position. Après avoir attendu un certain temps des réponses, l'agent recalcule sa position via un GPS ou l'algorithme Vivaldi. Puis applique l'algorithme de nuée, à partir des positions des voisins reçues précédemment, afin de décider de son déplacement.

À chaque tic d'horloge locale, les agents recalculeront leur position et essaieront de se déplacer en nuée. Les agents ne broadcasteront pas forcément à chaque tic d'horloge mais dépenderont de la fréquence de communication spécifié pour chaque agent. Lorsqu'un agent ne communique pas, il ne considère que les agents qu'ils connaissaient lors de la dernière communication et suppose qu'ils suivent le même chemin.

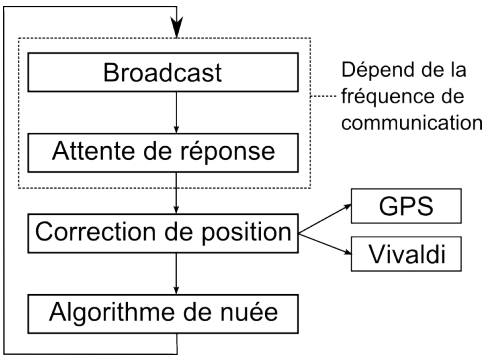

FIGURE 3.1 – Fonctionnement d'un agent

4 | Tests et résultats

4.1 Cadre expérimental

La durée d'exécution des tests est très importante, en effet, si tous les tests s'étaient déroulés sur un machine, il aurait fallu plusieurs centaines d'heures de calcul. Pour résoudre ce problème, nous avons effectués les tests sur plusieurs machines libres services de l'université, ainsi que sur notre machine personnelle.

Plusieurs séries de tests ont été effectués afin d'observer l'influence de certains paramètres sur l'ensemble du système.

Les paramètres testés sont

- l'influence de la précision du GPS ;
- le temps de formation des nuées.

De plus nous observerons les effets d'une initialisation par GPS sur l'algorithme Vivaldi.

Pour ces tests, 25 agents ont été utilisés lors des simulations afin de ne pas surcharger l'environnement, ce qui risquait de biaiser les résultats.

Chaque agent ne communique avec les autres qu'une fois tout les 10 tics d'horloge (locale), ceci produisant des résultats de cohésion et collision très similaire à une communication continue (voir figure 4.5), cela permet donc de réduire le temps d'exécution.

4.2 Métriques

Nous observerons deux métriques différentes

- le nombre de collision ;
- la cohésion de la nuée.

4.2.1 Collision

On considère qu'une collision se produit lorsque deux agents ou plus sont trop près les uns des autres. C'est à dire quand la distance entre ces agents

est inférieure à un seuil prédéfini. La valeur de collision correspond ainsi au pourcentage d'agents présent dans une collision.

4.2.2 Cohésion

La cohésion est la capacité des agents à se regrouper. Deux agents sont considérés dans le même groupe lorsqu'ils peuvent communiquer directement ou indirectement, via l'intermédiaire d'un ou plusieurs autres agents.

La mesure de cohésion est donc le pourcentage d'agents dans le plus grand groupe.

4.3 GPS non bruité

Tout d'abord, afin d'avoir un point de comparaison entre les différents tests, une série de tests utilisant un GPS parfait, sans erreur, sans bruit a été réalisé de manière à posséder les résultats idéaux, ou chaque agent aurait une connaissance parfaite de sa position.

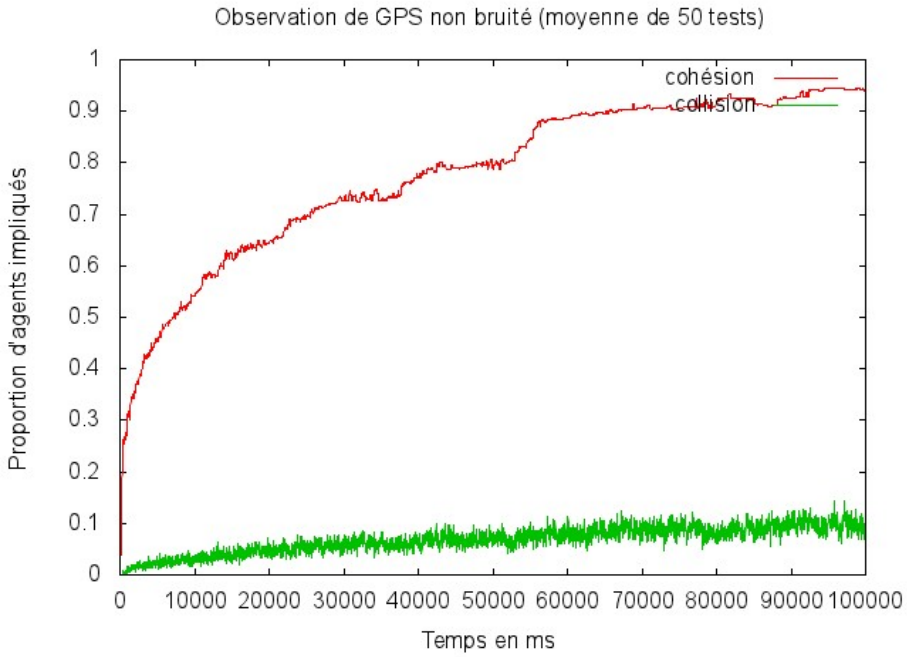

FIGURE 4.1 – GPS non bruité : cohésion et collisions
De haut en bas : la cohésion et les collisions.

Il est donc possible d'observer que la cohésion de la nuée augmente au fil du temps et fini par englober la majorité des agents, mis à part quelques agents s'échappant par moments de la nuée ,mais finissant toujours par y revenir. De plus le nombre de collision reste relativement faible puisqu'à peine 10% des agents sont impliqués, ce qui correspond à seulement une collision entre deux agents.

4.4 GPS bruité

Il était souhaitable de voir l'influence des parasites, du bruit, sur un GPS, afin de savoir si Vivaldi pouvait être plus efficace qu'un GPS peu précis. L'intensité du bruit correspond a l'écart type d'un loi normale de moyenne 0, qui est ajouté à la valeur du GPS.

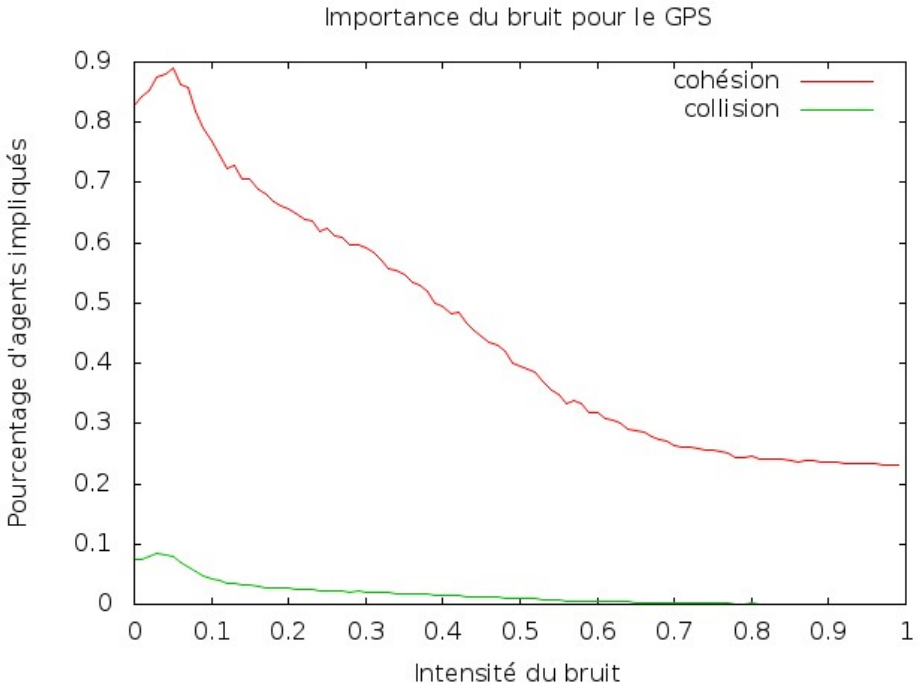

FIGURE 4.2 – GPS bruité : cohésion et collision
De haut en bas : la cohésion et les collisions.

De manière évidente, la cohésion et le nombre de collisions chutent à mesure que le bruit augmente, en effet les agents se détectent de moins en

moins bien et planifient donc un mouvement de nuée avec un agent dont la position pourra être modifiée drastiquement à l'instant suivant. Cela aura aussi pour conséquence de donner des vecteurs de déplacements des autres agents totalement erronés et fortement disproportionés.

4.5 Algorithme de Vivaldi

Il fallait ensuite observer les effets du système de positionnement Vivaldi comme unique méthode de positionement, au cour du temps.

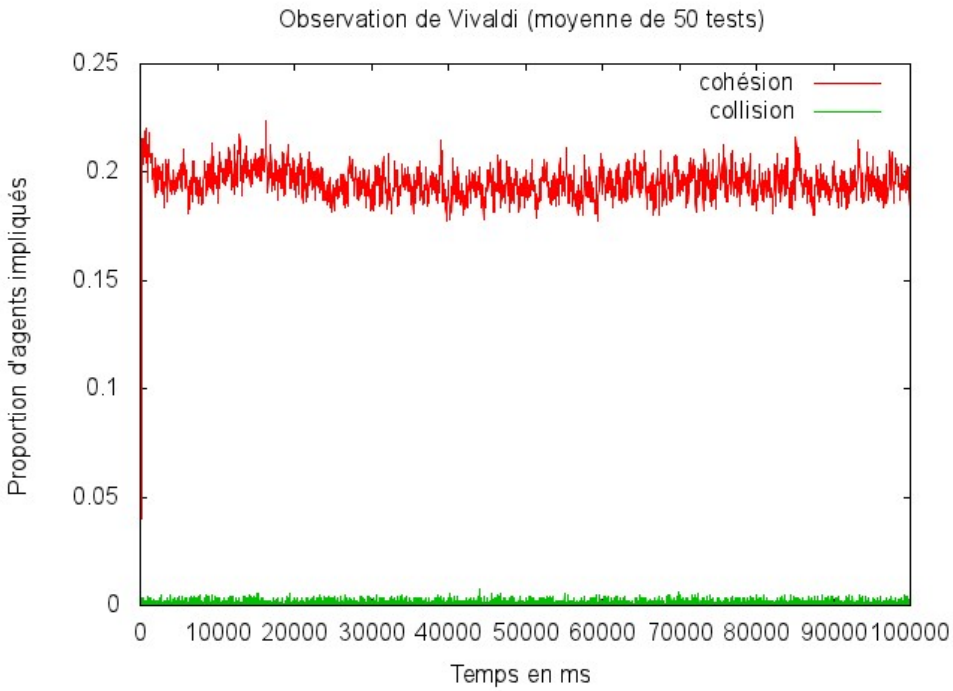

FIGURE 4.3 – Vivaldi : cohésion et collisions
De haut en bas : la cohésion et les collisions.

Il ressort donc que Vivaldi ne permet pas de maintenir la cohésion dans un algorithme de nuée. Ceci est visible au vu de la cohésion constante aux alentours de 20%, ce qui est du à la répartition des agents dans l'environnement. De plus le faible pourcentage de collision implique que les agents sont relativement séparés.

On peut donc en conclure que l'algorithme n'est pas utilisable tel quel pour maintenir la cohésion d'un algorithme de nuée.

La figure 4.4 montre bien les différences entre la nuée utilisant le GPS et celle utilisant simplement l'algorithme Vivaldi.

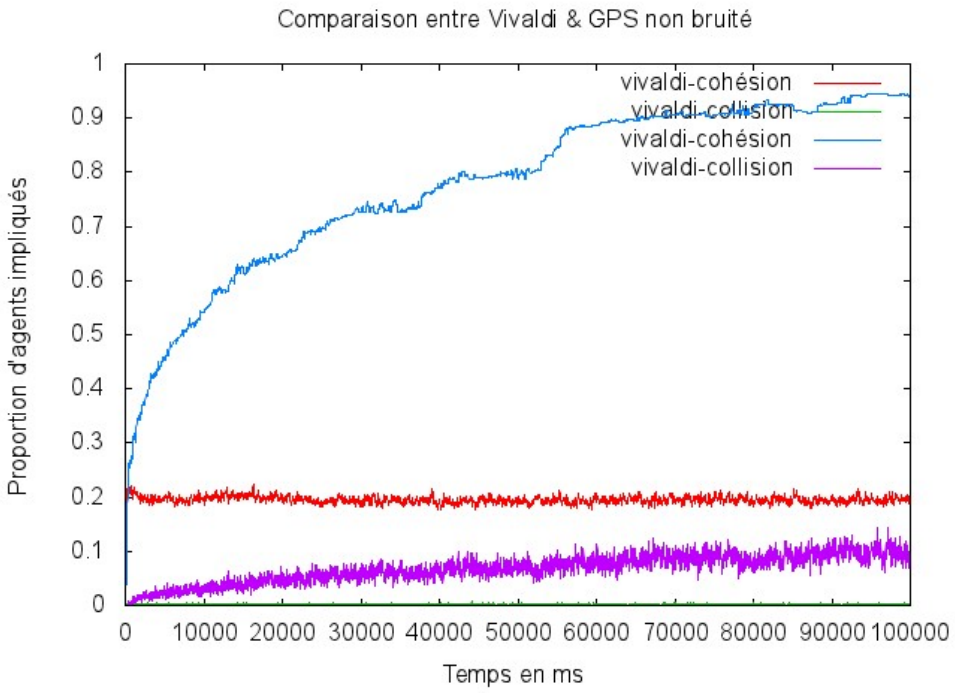

FIGURE 4.4 – Comparaison des collisions et cohésions entre GPS et Vivaldi De haut en bas : cohésion du GPS, cohésion de Vivaldi, collisions du GPS, collisions de Vivaldi.

4.6 Influence de la fréquence de communication

Enfin la fréquence de communication des agents peut être un point très important quant au maintien de la cohésion dans la nuée.

Il ressort donc de la figure 4.5 que la fréquences de communication n'a pas d'effet visible sur l'utilisation d'un GPS ou de l'algorithme Vivaldi, puisqu'en moyenne la cohésion de chacune de ces méthodes est identique aux différentes fréquences étudiées.

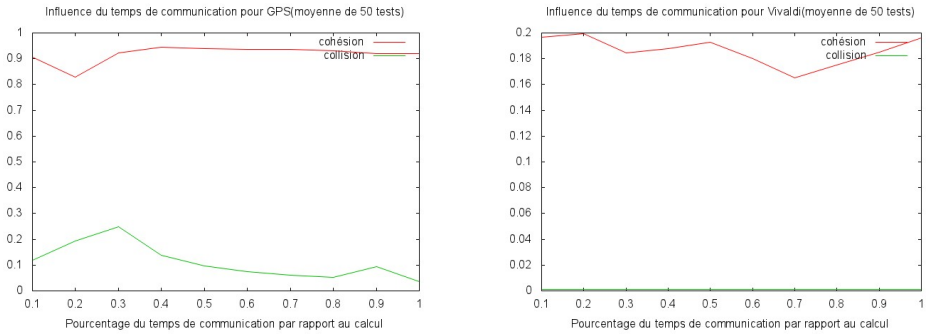

FIGURE 4.5 – Influence de la fréquence de communication sur le GPS et Vivaldi

À gauche les résultats avec le GPS , à droite avec Vivaldi.

Seul le nombre de collision diminue lorsque la fréquences de communication augmente.

4.7 Vivaldi précédé par un GPS

Afin de savoir si l'initialisation des positions artificielles des agents a une influence sur l'efficacité de l'algorithme Vivaldi, nous commencons les tests par l'utilisation d'un GPS, afin que tous les agents puissent initialiser de manière précise leurs positions artificielles, puis nous passons sur Vivaldi. L'arrêt de l'utilisation du GPS se fait au alentour de 20000 ms.

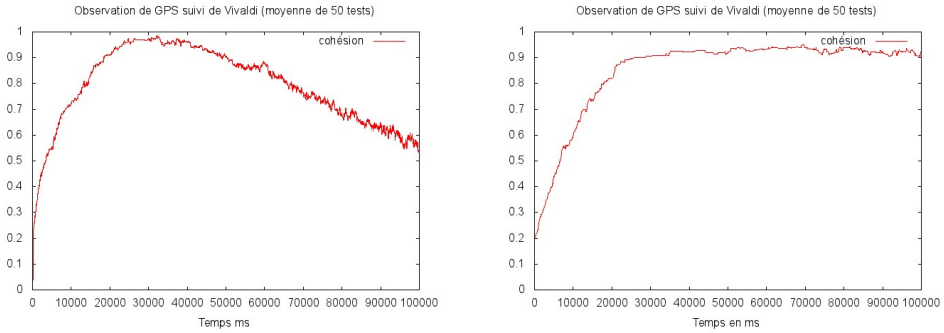

FIGURE 4.6 – Algorithme de Vivaldi précédé par un GPS.
À gauche fréquence de communication faible, à droite fréquence de communication maximum.

On peut donc remarquer qu'en initialisant l'algorithme Vivaldi par l'utilisation d'un GPS, l'algorithme Vivaldi permet de maintenir la cohésion dans la nuée, de manière plus efficace. Cependant l'apparition d'erreurs dans les calculs diminue rapidement la qualité de la nuée et rend l'algorithme de Vivaldi inefficace lorsque les communications sont peu fréquentes. L'augmentation de la fréquence des communications permet d'augmenter le maintien de la cohésion. Lorsque la fréquence de communication est à son maximum (figure 4.6 à droite), l'algorithme de Vivaldi permet de maintenir la cohésion de manière très efficace pendant une longue durée.

Il semblerait que la valeur de cohésion au cours du temps, en fonction de la fréquence de communication, puisse être représentée par une fonction

$$f(temps, fréquence) = cohésion.$$

5 Conclusion

5.1 Problèmes

Outre les problèmes d'implémentation de l'application, il fut difficile de trouver des tests pertinents à réaliser afin de vérifier le bon fonctionnement de l'application, ainsi que de trouver une façon de faire fonctionner efficacement l'algorithme de Vivaldi avec un algorithme de nuée.

5.2 Résultats

Plusieurs résultats sont apparus lors de ce projet.

Tout d'abord, l'algorithme de Vivaldi seul ne permet ni de maintenir ni de créer de nuées.

Ensuite, nous avons remarqué qu'il est possible de maintenir des nuées avec Vivaldi lorsque les positions sont initialisés via un GPS, pour une durée dépendente de la précision de ce dernier.

5.3 Limites du projet

Par manque de temps, il n'a pas été possible de tester convenablement l'application notamment quant aux conditions nécessaire pour maintenir la cohésion dans les nuées.

De ce fait, il n'a pas été possible de mesurer le gain énergétique de la réduction de l'utilisation du GPS au profit de l'algorithme Vivaldi.

5.4 Perspectives

Nous pourrions envisager de tester diverse méthodes afin de pouvoir faire fonctionner de manière durable l'algorithme de Vivaldi en maintenant la cohésion, sans avoir besoin d'utiliser de GPS sur les agents.

Pour cela, dans un premier temps, nous pourrions rajouter des agents utilisant uniquement un GPS, se signalant comme tel ou non, afin d'influencer l'algorithme Vivaldi à trouver une cartographie correcte des agents.

Ou bien, le fait d'installer des points fixes, des tours relais connaissant leurs positions, afin qu'elles le signalent aux agents, toujours dans le but d'influencer l'algorithme Vivaldi.

5.5 Bilan

Ce projet fut l'occasion de s'initier à travail de recherche, ainsi qu'à son application possible dans une situation réelle.

De plus cela m'aura permis de découvrir, d'utiliser et de créer des systèmes multi-agents

A | Mode d'emploi

A.1 Lancement

Notre application se lance via la ligne de commande grâce à la commande :

```
java -jar nuee.jar [options]
```

Elle génèrera automatiquement des fichiers de données dans un dossier, par défaut le dossier `data` du répertoire courant, si il existe.

La taille de l'environnement est de 1000 par 600 unités.

A.2 Options

Les différentes options de lancement sont

–**no-ui** Désactive l'interface graphique.

-na | –**number-of-agents** Spécifie le nombre d'agents initiaux, par défaut 20.

-d | –**duration** Spécifie la durée du test (en ms), par défaut 3mn.

-p | –**position-method** Choix de la méthode de positionnement : 0 (GPS) ou 1 (Vivaldi), par défaut 0.

-c | –**communication-dist** Distance de communication maximale entre agents, par défaut 100

-s | –**separation-dist** Distance de séparation pour la nuée, par défaut 33.

-cs | –**communication-speed** Fréquence de communication, par défaut 1.0.

–**speed** Vitesse des agents, par défaut 10 unités/tic.

–**seed** Graine des générateurs, par défaut 123456789

-o | –**output** Dossier de sauvegarde des fichiers de données.

A.3 Interface graphique

Si l'option –**no-ui** n'est pas spécifiée, l'application sera lancée avec une interface graphique, composée de deux fenêtres visibles ci-après.

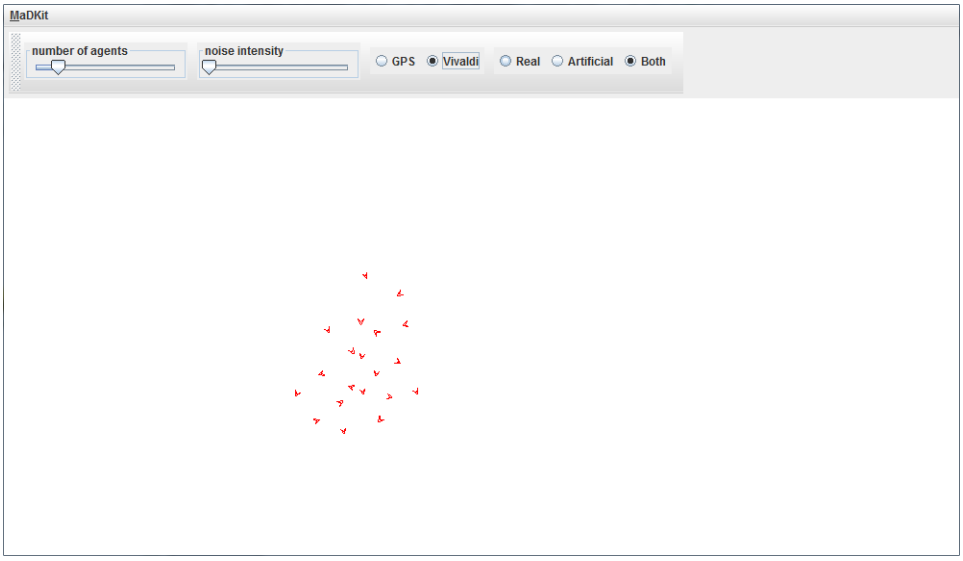

FIGURE A.1 – Fenêtre principale de l'application

Si l'option –**no-ui** est spécifiée, l'application n'aura pas d'interface graphique, et les tics d'horloge seront plus fréquent.

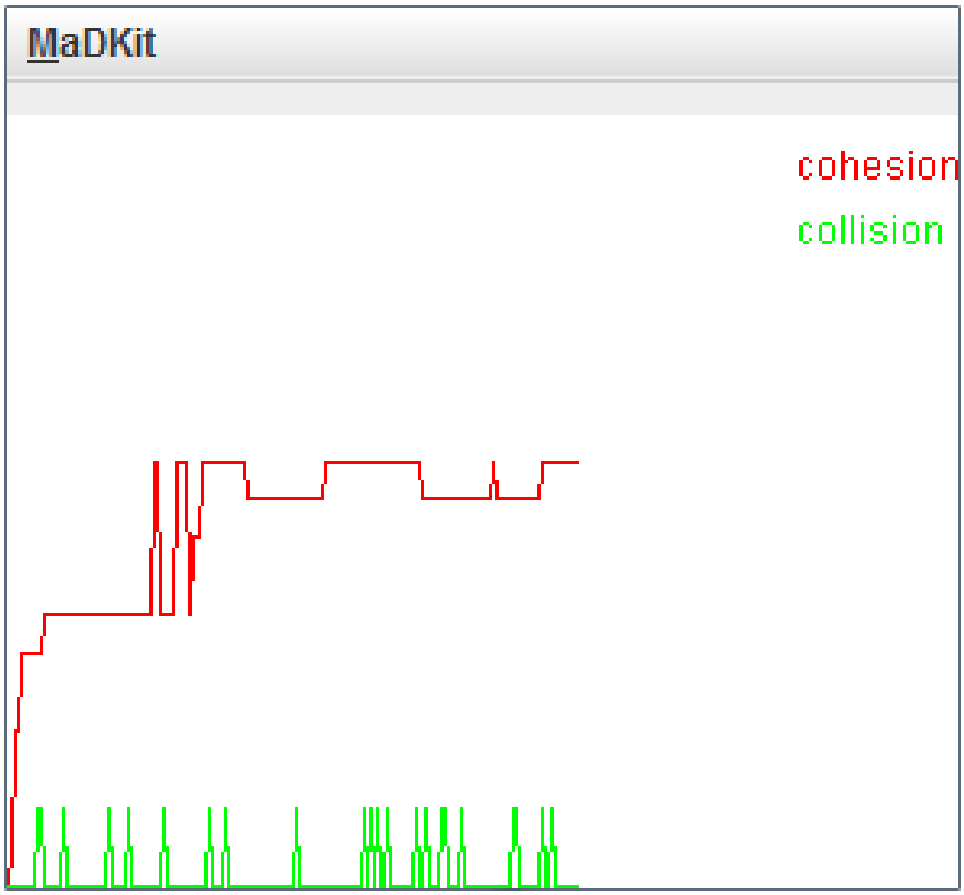

FIGURE A.2 – Exemple d'execution de l'algorithme de vivaldi

B | Diagramme UML

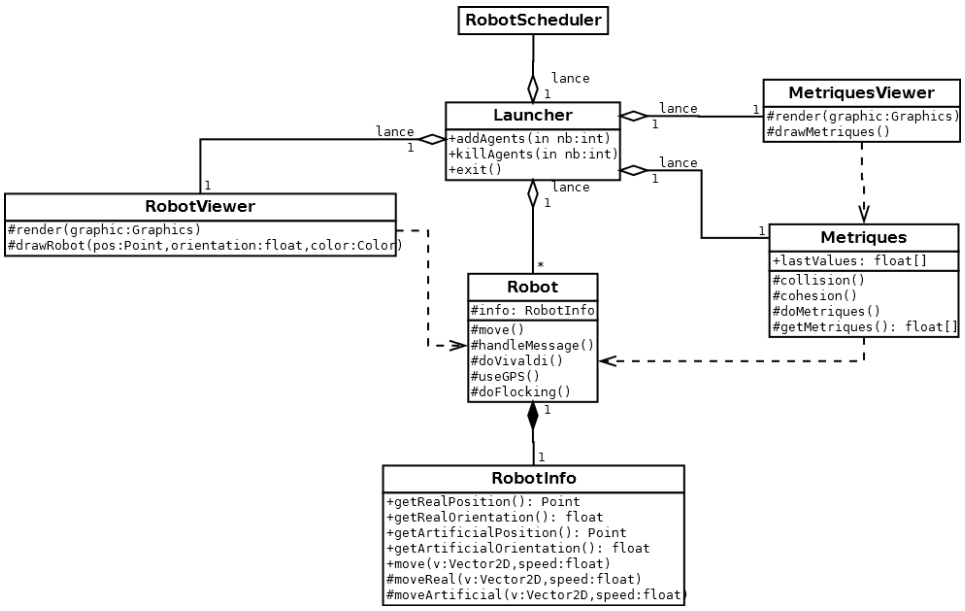

Figure B.1 – Diagramme UML de l'application

Bibliographie

[1] F. Dabek, R. Cox, F. Kaashoek, R. Morris. Vivaldi : A Decentralized Network Coordinate System, MIT CSAIL, Cambridge MA, August 2004.

[2] MaDKit http://www.madkit.net/

[3] C. W. Reynolds. Flocks, Herds, and Schools : A Distributed Behavioral Model, July 1987 et http://www.red3d.com/cwr/boids/.

[4] Swarm behavior - Wikipedia http://en.wikipedia.org/wiki/Swarm_behaviour

[5] Modèle Agent/Groupe/Rôle - J. Ferber http://www.gemas.fr/dphan/PorquerollesXAgents/cours/JFerber-_part2.pdf